

Evaluation of quadrat-related functions

Gabriel Arellano

2026-05-30

Contents

1	Goal	1
2	Load code	2
3	Map dependencies	3
4	Evaluation	5
4.1	Goal and structure of most functions	5
4.2	Neighboring-related functions	7
4.3	Other functions	7
4.4	Depending functions	7
5	Design principles for new code	8
5.1	Not urgent	9
6	Clean up	10

1 Goal

There is a set of functions in the CTFS / fgeo functionality that seems related to handling quadrats and coordinates in ForestGEO plots. Most of the 20 functions seem to translate from one form (and resolution) of location into another. These are the types of location information in this type of datasets:

- gx and gy: global coordinates within the plot, relative to a (0, 0) corner
- lx and ly: local coordinates within the quadrat
- col and row: column and row within the plot

- conceptually, just coarse x and y coordinates
- in practice, could be named with letters, start at 00 or 01, etc.
- quadrat name, following certain conventions, typically derived from row and col
- hectares (rarely)
- quadrat index, numerical
- quadrats of different resolution than the default
 - typically smaller quadrats nested within larger quadrats
 - sometimes with specific naming that refers to location within larger quadrat (e.g. 5x5 m quadrats, useful for fieldwork and filling maps, within 20x20 quadrats used for data analyses and summaries)

Additionally, there are functions related to identifying neighbors, selecting quadrats, and other related tasks. Probably secondary in importance.

This is the evaluation of the functions in the CTFs `quadfunc.r` script and, possibly, their downstream dependencies.

2 Load code

We declare paths to the project (watched by git), load a specific version of the code, and auxiliary functions.

```
# This is my local path to the project (being watched by git)
# You may have to adapt this.
path <- normalizePath(system2("git", c("rev-parse", "--show-toplevel"), stdout = TRUE)[1])
knitr::opts_knit$set(root.dir = path) # global wd, instead of within-chunk wd as setwd()

# Source useful functionality:
source(paste0(path, "/evaluation/0-code-to-source.r"))

# For reproducibility, this file does code evaluation at a specific
# point in time. Otherwise it won't make sense as the project
# moves forward and the code gets updated.

# The following lines create a temporary file and gives its path.
# You have to clean that at the end of the file, otherwise
# you will accumulate temporary folders somewhere.

DATE = "2026-05-29"
commit = commit_from_date(path = path, date = DATE)
historical_path <- make_dir_to_point_in_history(path = path, commit = commit)
```

```
# Other functionality:  
library(igraph, quietly = TRUE)
```

Warning: package 'igraph' was built under R version 4.4.3

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

This will source the necessary scripts; note that I am loading a specific version of them, by using the path to a specific commit (`historical_path`, linked to commit = 8b1c13366dba84a757fb08ebd9cf5d813c0d1668).

```
modules <- c("01_utilities", "02_biomass", "03_spatial", "04_abundance_diversity",  
            "05_demography_growth", "06_topography_habitat", "07_mapping_plotting")  
  
scripts <- unique(unlist(lapply(modules, function(m) {  
  list.files(paste0(historical_path, "/", m, "/R/new"),  
            full.names = TRUE, pattern = "\\\\.R$", ignore.case = TRUE)  
})))  
  
# Source scripts, then new:  
for(f in scripts) source(f)
```

3 Map dependencies

This maps the mutual dependencies among the functions:

```

# All of the dependencies among all functions in the global environment:
edges <- get_all_dependencies()

# Identify the target subset of functions and their dependents:
where <- grep("quadfunc.r", scripts)
quadrat_functions <- functions_in_script(scripts[where])
down <- unique(unlist(lapply(quadrat_functions, function(q) downstream_dependents(q, edges))))
down <- down[!down %in% quadrat_functions]
target <- c(quadrat_functions, down)

# Graph representation:
keep <- edges$from %in% target | edges$to %in% target
edges <- edges[keep,,drop = FALSE]
g <- graph_from_data_frame(edges, directed = TRUE)

# Visualize
V(g)$color <- rep("gray", length(V(g)))
V(g)$color[V(g)$name %in% quadrat_functions] <- "gold"
V(g)$color[V(g)$name %in% down] <- "orange"

plot(g, layout = layout_fruchterman_reingold,
     vertex.label = V(g)$name,
     vertex.color = V(g)$color,
     vertex.frame.color = "white",
     vertex.label.color = "black",
     edge.color = "black")

```


location:

- `gx` and `gy`: global coordinates
- `lx` and `ly`: local coordinates
- quadrat name
- quadrat index
- row and column (of quadrats)

There are somewhat cryptic mentions to smaller quadrats in the names of `getsmallerquads` and `lxly.to.p5` functions. No functions depend on them, so it is possible that they will be removed or substituted by more general ways of splitting into quadrats of arbitrary resolution.

It seems most of the difficult things require quadrat indexes, which is not something users will be familiar with, as the data tables typically store quadrat names, the fieldwork is done using quadrat names, etc. This, in itself, makes anything based on quadrat indexes difficult to de-bug or scrutinize, and thus it is a point of vulnerability.

The functions apply to trees or quadrats, with quadrats being simplified as points. For example:

- `quad.to.gxgy`: returns global coordinates of the bottomleft coordinate of each quadrat, given a CCRR column-row naming convention, and a given gridsize (= quadrat size).
- `gxgy.to.quad`: returns the quadrat membership of the trees, given their global coordinates.

For the CCRR column-row naming convention, sometimes `start = 0`, sometimes `start = "zero"`. This refers to whether the first column (or row) is indicated by "00" or "01". This, by the way, should be trivial to guess from the complete data table.

Interestingly, the `index.to.rowcol` function returns numerical rows and columns, that always start with 1. So we can find datasets with `row = 1`, `col = 1`, and `quadrat = 0000` in CCRR convention with `start = 0`.

The CCRR using numerical digits is a typical point of vulnerability in ForestGEO datasets, as it looks like a number, but it is not a number. Excel often goes from "0133" to "133". This creates issues again and again. Condit code has warnings related to that, things like "Must rewrite if three digit quadrats". The function `convert.rowcol` adds initial zeros.

There is a weird convention in Condit's code when coordinates are unknown, where `quadrat = 9999` instead of `quadrat = NA`. In contrast, `gxgy.to.index` returns `quadrat index = NA` when there is not enough information to determine location.

4.2 Neighboring-related functions

There is a small set of functions related to shifting, identifying and comparing with neighboring quadrats... Possibly related to null models:

- `create.neighbordata`: risky and confusing. Takes as input an “abundanceperquad” object, which follows certain conventions (species in rows, data sorted by quadrat index). Clear candidate for removal. Nothing depends on this. The input is probably the output of the `abundanceperquad` function, but the dependency is implicit, not explicit. It uses `findneighborabund` (next).
- `findneighborabund`: takes similar input, but called “abundvect” now. It finds neighboring quadrats using `findborderquads` (next), and then copy-pastes abundances from the abundance vector.
- `findborderquads`: calculates indexes of neighboring quadrats, for a given quadrat index. Probably useful, although using this just to calculate species abundances seems like a narrow application.
- `neighbors`: very bad name, super general word, for an extremely narrow application. This function takes a vector of absence/presence for all quadrats, and returns proportion of presences among the neighboring quadrats. It is difficult to understand why this has become part of the broadly distributed CTFS package.
- `torus.shift`: Creates a torus-shifted quadrat topographic dataset. This can be safely removed if a standard torus translation (or constrained, for that matter) is integrated into the pack of functions. Any quadrat-based map can be translated just by translating the corners, then going back to (null) quadrat codes, if needed.

Although not directly connected, there is an interesting function `distance()` that calculates the distances between quadrats. It only wraps from quadrat indexes to coordinates, and then applies `xydistvect()` function in `utilities > geometry`. Using distances seems a more general approach to identify neighboring quadrats, generalizable to $k = 4$, $k = 8$, and so on, using standard k -NN routines.

4.3 Other functions

`getsmallerquads()`: this is a dead-end function, nothing depends on it. It returns small quadrats within larger quadrats, given two resolutions. The smaller resolution will be adjusted internally. It seems a rather arbitrary sub-division of quadrats, instead of directly dividing the entire plot (or any point dataset, for that matter) into quadrat membership at any resolution.

4.4 Depending functions

There are different pats of CTFS / fgeo code that depend on the quadrat-related functions.

First, they relate with `fullplot.imageJ`. That function is part of a practical routing to go from paper maps to digital coordinates. Still waiting to understand whether this is needed now that people work with tablets and all that.

Second, `gxgy.to.index` and `index.to.gxgy` relate to, apparently, rather trivial functions to pick quadrats or summarize abundance by quadrat: `abundanceperquad` and `selectrandomquad2`. Clearly, these tasks don't need dedicated functions. The user must be able to summarize abundances by any combination of things, e.g. [quadrat x species] or [family x size class x habitat], or whatever. Something similar can be said for `selectrandomquad2`.

Third, `gxgy.to.index` is called by functions that run the wavelet analysis applied to Forest-GEO datasets. This is probably something work isolating and developing by itself. Ideally, it should be generalize and integrated with other spatial null models. In any case, it does not seem a desirable or necessary dependency.

Fourth, there are links to a topographical function: `allquadratslopes`.

Finally, there is an isolated pair of functions that should be examined together:

- `full.xygrid` (in the `quadfunc.r` script) is equal to `base::expand.grid()`.
- `graph.mvnorm` (in `utilities > distributions`) apparently is just used to graph contours on multivariate normal distributions). Maybe something to move to the mapping + plotting module.

5 Design principles for new code

Clearly, the only piece of information really needed is just the global coordinates, $\{gx, gy\}$. That is what should be kept in a good, easy-to-use data table.

However:

- that is not always the case
- during data digitization the input can be local coordinates and quadrats, so we still need functionality to end with global coordinates.

If $\{gx, gy\}$ is the stepping stone, then we can split these translations as two separated problems:

- from whatever you have (quadrat names from field, local coordinates at 20x20 m or 5x5 m resolution, etc) into global coordinates.
- from global coordinates into whatever way of splitting the plot you want:
 - specific or customized, to go back to specific quadrat naming conventions
 - general: quadrats of any resolution

- * there will be unused borders, remainders
- * some resolutions will fit exactly, we can anticipate them

A very general wrapper would be: [anything you have] → [everything you may possibly need]. And then tell the user to keep everything in the table and call the appropriate column whenever is the case. However, this would be sometimes limited. For example:

- we can randomize habitat membership by randomizing quadrats, assuming a habitat map as a {quadrat, habitat} dataset.
- we can randomize global coordinates and then access other datasets through quadrat membership
- etc.

Here I am paying attention to code, not data conventions. The [5x5 m within 20x20 m quadrats] convention is something that ForestGEO users must be able to navigate with the new code. That means respect towards `lxly.to.p5` function.

The neighbor-related functions, according to the “functions cannot be examples” principle, will be probably substituted by:

- A function to identify neighboring quadrats
 - possibly constrained (e.g. only neighbors of the same habitat, or same community type, etc., using arbitrary grouping)
 - possibly through a “distance between quadrats” intermediate step, so we can generalize to $k=4$, $k=8$, etc., using k -NN routines.
- Tutorials or examples: abundances, presence/absence, basal area, ...

As happens with the neighborhood analyses (of points): neighboring quadrats are just pieces of forest, subsets of trees, and there are many things that could be done with that, not just a couple.

It seems that a “split into quadrats of side = x” function in the script will be useful, in general:

- if applied to the entire plot, and fits well within 20 m, then it will do what `getsmallerquads()` does
- if not, then it’s just two classifications into non-nested quadrats
- if can be applied to local or global coordinates, so this could also do what `lxly.to.p5()` does.

5.1 Not urgent

- quadrats are just polygons... Perhaps there are gains if we facilitate a polygon representation for quadrats.

6 Clean up

```
# Remove the temporary folder we created to
# access the project at a specific point in history:
knitr::opts_knit$set(root.dir = path)
if(dir.exists(historical_path))
  remove_dir_to_point_in_history(path, historical_path)

dir.exists(historical_path) # should be FALSE now
```

```
[1] FALSE
```